NOVEL-TO-MOVIE AI

Functional Requirements Document (FRD)

System behaviour, workflows, data model and acceptance criteria

Project	AI-powered multimodal story-to-film platform
Target event	Google AI Builder Cup 2026 hackathon
Document	FRD • NOVEL-TO-MOVIE-AI
Version	1.0 — Draft for implementation
Date	12 October 2026

Working principle

Turn a user-owned short novel into a coherent animated film through story understanding, screenplay planning, character and scene design, short-form video generation, audio, and automated editing.

Scope note: The 15/30/45/60-minute options are target runtimes for a future full pipeline. A hackathon MVP should prove the end-to-end workflow with a short sequence or trailer before attempting feature-length generation.

1. Purpose and system boundary

This FRD defines the observable behaviour and interfaces required to implement the Novel-to-Movie AI MVP. It is derived from the PRD and focuses on what the system must do, including validation, asynchronous processing, failure recovery, persistence and export.

System boundary: the application orchestrates user uploads, Gemini story reasoning, image/video generation APIs, asset storage and FFmpeg assembly. Foundation models are external services; the application does not train its own model.

2. Roles and permissions

Role	Permissions
Guest / user	Create a project, upload/paste a story, configure settings, approve/edit screenplay, approve visual references, generate scenes, review and export own project
System worker	Process queued jobs, call configured model APIs, persist outputs and statuses, retry eligible failures, assemble export
Administrator (minimal)	Inspect operational errors and usage logs; configure model IDs and limits through environment/configuration, not public UI

3. Functional requirements

Priority key: P0 = required for the demo; P1 = important if time permits; P2 = later enhancement.

ID	Priority	Requirement	Acceptance criteria
FR-001	P0	Create project	User can create a project with title and language; system returns a unique project ID.
FR-002	P0	Ingest story	User can paste text or upload a supported text file; unsupported formats and oversized inputs produce a clear validation message.
FR-003	P0	Configure adaptation	User can select genre/tone, animation style, aspect ratio, language and a preview runtime preset.
FR-004	P0	Analyze story	System returns a validated story bible: synopsis, characters, relationships, settings, timeline, plot beats and themes.
FR-005	P0	Generate screenplay	System creates ordered scenes with narrative purpose, location, characters, action, dialogue/narration, visual direction and target duration.
FR-006	P0	Edit and approve plan	User can edit scene text and approve the screenplay before visual/video generation begins.
FR-007	P0	Generate visual references	System can request character/setting reference images; each asset is linked to a project and character/location.
FR-008	P0	Approve references	User can approve or request regeneration of a reference before production.
FR-009	P0	Start production	System estimates clip count/usage, asks for confirmation and queues scene-generation jobs within configured caps.
FR-010	P0	Generate video clips	Worker calls configured video model, records operation/job status and stores each successful clip.
FR-011	P0	Track progress	UI displays project and per-scene states, including queued, generating, completed, failed and needs review.
FR-012	P0	Retry failed scene	User can retry a failed scene without rerunning completed scenes; retries obey limits and budget caps.

ID	Priority	Requirement	Acceptance criteria
FR-013	P0	Assemble preview	System orders selected completed clips and uses FFmpeg to assemble a playable MP4; missing scenes are clearly indicated.
FR-014	P0	Preview/download	User can play the assembled preview and download the output when assembly succeeds.
FR-015	P0	Persist project	Project settings, screenplay version, scene states, asset references and export state survive page refresh.
FR-016	P1	Narration/subtitles	System can add narration and/or subtitle tracks where the chosen providers and timeline support it.
FR-017	P1	Regenerate selected scene	User can change a scene prompt and regenerate only that scene, preserving prior versions.
FR-018	P1	Usage accounting	System records model, request count, duration, status and estimated/actual usage where exposed.
FR-019	P2	Long-form runtime	System supports multi-batch production toward 15/30/45/60-minute targets subject to quotas, cost caps and operational limits.
FR-020	P2	Advanced timeline editing	User can reorder clips, trim boundaries and adjust audio/subtitles before export.

4. End-to-end workflow and state model

Happy path: Create project → ingest story → analyze → review story bible → generate screenplay → edit/approve → generate references → approve references → estimate and confirm production → queue scene jobs → generate clips → review/retry → assemble → preview/download.

Project states: DRAFT, INGESTING, ANALYZING, PLAN_REVIEW, DESIGNING, DESIGN_REVIEW, READY_TO_GENERATE, GENERATING, SCENE_REVIEW, ASSEMBLING, COMPLETED, FAILED, CANCELLED.

Scene states: PLANNED, QUEUED, GENERATING, GENERATED, NEEDS_REVIEW, APPROVED, FAILED, RETRY_QUEUED, OMITTED.

State changes must be persisted. A worker restart must not silently reset completed work. Repeated user submissions should be idempotent where practical, especially for starting a production job.

5. Story-analysis output schema

Gemini should be instructed to return schema-constrained JSON where supported. Validate the response server-side with Pydantic before storing it. Never rely on free-form prose parsing for critical workflow fields.

Object	Required fields (illustrative)
StoryBible	title, logline, synopsis, genre, tone, themes[], characters[], locations[], plot_beats[], continuity_rules[]
Character	character_id, name, role, age_range_or_appearance, personality_traits[], wardrobe, visual_description, relationships[]
Location	location_id, name, description, time_of_day_options[], visual_motifs[]
Scene	scene_id, order, title, narrative_purpose, location_id, character_ids[], action, dialogue_or_narration, visual_prompt, continuity_notes, target_seconds, status
ProductionSettings	style, language, aspect_ratio, target_runtime_seconds, preview_mode, max_clips, budget_limit

6. Backend API contract (proposed)

Method & path	Purpose	Success response
POST /api/projects	Create project	201 with project_id and initial state
POST /api/projects/{id}/story	Upload/paste story text	202 or 200 with ingestion status
PATCH /api/projects/{id}/settings	Update settings	200 with validated settings
POST /api/projects/{id}/analyze	Run story analysis	202 with job_id
GET /api/projects/{id}	Fetch project and current state	200 with project metadata and state
GET /api/projects/{id}/story-bible	Fetch story bible	200 with validated story bible
POST /api/projects/{id}/screenplay	Generate/re-generate screenplay	202 with job_id
PATCH /api/projects/{id}/scenes/{scene_id}	Edit scene plan	200 with updated scene/version
POST /api/projects/{id}/approve-plan	Approve screenplay	200 with project state
POST /api/projects/{id}/references	Generate reference assets	202 with job_id
POST /api/projects/{id}/approve-references	Approve visual assets	200 with project state
POST /api/projects/{id}/production	Estimate/confirm/start production	202 with production_id and queued scenes
GET /api/projects/{id}/jobs	Poll job and scene status	200 with progress summary
POST /api/projects/{id}/scenes/{scene_id}/retry	Retry one failed/selected scene	202 with job_id
POST /api/projects/{id}/assemble	Assemble selected clips	202 with assembly_job_id
GET /api/projects/{id}/exports	List export status and URLs	200 with export metadata

7. Data model

Entity	Key fields	Relationship / purpose
Project	id, title, owner/session_id, source_text_ref, settings_json, state, created_at, updated_at	Top-level project record
StoryBible	id, project_id, version, json_payload, approved_at	Structured interpretation of story
Scene	id, project_id, order_index, plan_json, state, current_version	One planned narrative unit
Asset	id, project_id, scene_id nullable, asset_type, storage_uri, model_id, prompt, status, created_at	Image/video/audio/subtitle output
Job	id, project_id, scene_id nullable, job_type, state, attempts, provider_operation_id, error_code, timestamps	Async work and recovery
Export	id, project_id, storage_uri, duration_seconds, state, created_at	Assembled output
UsageEvent	id, project_id, job_id, model_id, request_metadata, estimated_cost, actual_cost nullable	Budget and operational monitoring

8. Model orchestration requirements

- Use the official Google Gen AI SDK and server-side GEMINI_API_KEY environment variable. Never expose the API key in frontend code, browser storage or client logs.
- Use Gemini text/multimodal reasoning for story analysis and structured scene planning; use image-capable generation for references where available; use the configured Google video model for short clips.
- Keep model IDs in configuration so a model can be changed without changing business logic.
- Before development, verify API enablement, model ID, region availability, quota, pricing, content restrictions and response format support.
- Treat provider operations as asynchronous where applicable. Persist operation identifiers and poll with bounded backoff; do not keep a browser request open while generating a movie.
- Use stable character descriptions and approved reference images in scene prompts when supported; include prior-scene continuity summaries as text, not as a guarantee of exact visual identity.
- Validate all model-produced JSON; reject or repair schema-invalid output with a bounded retry. Never execute model output as code.
- Use provider safety responses as explicit scene-level failures and show a helpful message; do not endlessly retry policy-blocked content.

9. File upload and validation

- MVP supported formats: plain text (.txt) and pasted text; add PDF/DOCX only after extraction is tested.
- Enforce configurable byte and character limits; show the accepted limits in the UI.
- Reject empty, unreadable or unsupported files with a specific error.
- Store source text securely and avoid writing story contents to application logs.
- Prompt the user to confirm they have the rights or permission to adapt the submitted material.

10. Production budget and runtime estimation

Before production, estimate clip count using the selected target runtime and configured clip duration. For a target runtime T seconds and nominal generated clip duration D seconds, the theoretical minimum is ceil(T / D) clips; actual required clips may be higher due to retries, trims, transitions and rejected outputs.

Example at 8 seconds per clip: 15 minutes ≈ 113 clips; 30 minutes ≈ 225 clips; 45 minutes ≈ 338 clips; 60 minutes ≈ 450 clips. These are arithmetic lower bounds, not cost or completion-time promises.

For the hackathon, enforce a small maximum clip count and short preview runtime. Display a clear warning that long-form generation can take substantial time and incur substantial provider usage. Require explicit confirmation before launching expensive work.

11. Error handling and recovery

Condition	Required behaviour
Invalid input/file	Return 4xx with actionable validation message; do not create production jobs
Gemini schema-invalid output	Run bounded repair/retry; then mark analysis job failed and preserve error details safely
Rate limit / transient provider error	Retry with exponential backoff and jitter within configured limits
Safety rejection	Mark affected scene blocked/needs revision; explain that the prompt needs editing
Video generation timeout	Persist state, poll if provider supports operation retrieval, and allow later refresh
Storage/assembly error	Preserve completed clips; mark assembly failed and allow assembly retry
User retries scene	Create a new asset/job version; do not overwrite prior successful asset until replacement is accepted
Worker restart	Resume recoverable queued/running jobs using persisted job state and provider operation IDs

12. Non-functional requirements

Category	Requirement
Security	Keep API keys server-side; validate inputs; restrict asset access to the owning project; use HTTPS in deployment.
Privacy	Explain how uploaded stories and generated assets are stored and deleted; minimize logs and avoid logging full story text.
Reliability	Use persistent job states and bounded retries; a single failed scene must not destroy completed work.
Performance	Web APIs should return quickly for long-running tasks by enqueueing work and returning job IDs.
Scalability	Separate API from workers; use a queue/background worker if concurrent jobs grow.
Observability	Log project/job IDs, model ID, timings, retry counts and sanitized errors; expose useful progress to UI.
Accessibility	Use clear status labels, keyboard-accessible controls and readable contrast.
Maintainability	Use Pydantic models, modular provider adapters, environment-based configuration and tests for state transitions.

13. Security and privacy rules

- Load GEMINI_API_KEY from environment or a secret manager; fail startup safely if missing.
- Do not return secrets in API responses, exception messages or client-visible logs.
- Limit upload size and validate MIME type/extension; treat file content as untrusted.
- Use per-project authorization checks on every project, scene and asset endpoint.
- Define retention/deletion behaviour for source novels, generated assets and exports before public launch.
- Disclose that third-party model APIs process inputs under their applicable terms and settings; do not promise end-to-end confidentiality without verifying the provider terms.

14. Acceptance test checklist

Test ID	Scenario	Pass condition
AT-01	Create a project	Unique project appears and can be reloaded
AT-02	Submit empty/unsupported file	Helpful validation error; no analysis job queued
AT-03	Analyze valid short story	Valid story bible stored and displayed
AT-04	Generate scene plan	Scenes are ordered and required fields validate
AT-05	Edit a scene	Saved edit persists after refresh
AT-06	Approve screenplay	Video generation remains blocked until approval
AT-07	Start production over clip cap	Request is rejected with clear cap/estimate message
AT-08	Generate one clip	Job status and asset are persisted; preview is available
AT-09	Simulate provider failure	Failure is visible and retry is available where appropriate
AT-10	Retry one scene	Only selected scene is retried; completed scenes remain intact
AT-11	Assemble preview	FFmpeg output exists, plays and has expected clip order
AT-12	Download export	Authorized user can download completed export
AT-13	Inspect frontend bundle/logs	API key is not exposed
AT-14	Restart worker	Persisted jobs remain consistent and recoverable

15. Recommended implementation order

- **Step 1:** Verify Gemini API access and generate one short video clip manually with the intended model.
- **Step 2:** Implement FastAPI project creation, story input and Pydantic schemas.
- **Step 3:** Add Gemini story analysis and structured screenplay generation.
- **Step 4:** Build the review UI for story bible, scene list and character references.
- **Step 5:** Implement async job persistence, a bounded clip-generation worker and scene status updates.
- **Step 6:** Assemble clips with FFmpeg and provide a preview/download.
- **Step 7:** Add budget caps, retry controls, safety states, telemetry and a polished demo story.

16. External technical references

Gemini API — Veo video generation and model constraints: https://ai.google.dev/gemini-api/docs/veo

Gemini API documentation: https://ai.google.dev/gemini-api/docs

Treat model names, durations, quotas, pricing and availability as configuration-time facts to verify, not permanent assumptions.